

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XI
Error Handling



Disusun Oleh:
Danu Dimas Putra 105224003

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Dalam permintaan THT, sistem pemesanan tiket kereta api "JAVA EXPRESS" yang berfokus pada penerapan aplikasi yang stabil. Dalam penerapannya, program diminta untuk menangani pengguna salah ketik, misalnya memasukkan huruf padahal seharusnya angka, lalu program juga diminta untuk pembuatan menu pada tampilan sistemnya seperti melihat jadwal, memesan tiket, dan keluar dari sistem. Penerapan pada *blueprint* Kereta Api juga harus diawali dengan kosong pada setiap atribut nilainya (kode kereta, nama kereta, rute perjalanan, dan sisa kursi) namun ketika program berjalan, data/objek kereta otomatis ada dan tersedia.

Selanjutnya *blueprint* terkait pusat control, harus dapat melakukan perilaku/*method* pemesanan yang menerima input kode kereta, NIK, nama penumpang, dan jumlah tiket. Kemudian harus berjalan dengan validasi yang ketat disertai dengan penanganan error berupa melemparkan pesan error dari jenis *Unchecked* dan *Checked* yang berkaitan seperti *DataPenumpangTidakValidException* saat input berupa terdapat karakter huruf atau jumlah baris huruf tidak sama dengan 16, lalu *RuteTidakDitemukanException* saat input kode atau rute tidak ada, terakhir *TiketHabisException* saat jumlah tiket yang dipesan lebih dari sisa kursi yang tersedia. Pada *class* seperti main juga harus menangani error yang memanggil, seperti try-catch blok agar mencegah error berdasarkan pemanggilannya.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	kodeKereta	String	Untuk menyimpan huruf dari data kode kereta dan salah satu variabel yang berada sebagai atribut di dalam class.
2	namaKereta	String	Untuk menyimpan huruf dari data nama kereta dan juga salah satu variabel yang berada sebagai atribut di dalam class.
3	rutePerjalanan	String	Untuk menyimpan huruf dari data rute perjalanan dan juga salah satu variabel yang berada sebagai atribut di dalam class.
4	sisaKursi	int	Untuk menyimpan nilai angka bulat dari sisa kursi dan salah satu variabel yang berada sebagai atribut di dalam class.
5	daftarK	ArrayList<KeretaApi>	Variabel dengan ArrayList untuk menyimpan objek kereta api sebagai list kereta yang ada.
6	valid	KeretaApi	Variabel dengan tipe data KeretaApi yang ditujukan untuk menyimpan objek kereta api yang dicari/tersedia/ada.

7	namaKereta	String	Untuk menyimpan huruf dari data nama kereta dan juga salah satu variabel yang berada sebagai atribut di dalam class.
8	sisaKursi	int	Untuk menyimpan nilai angka bulat dari sisa kursi dan salah satu variabel yang berada sebagai atribut di dalam class.
9	c	char	Variabel perulangan pada foreach untuk memeriksa setiap karakter merupakan angka 0-9 (nik), selain angka maka gagal dan lempar exception.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	KeretaApi()	<i>constructor</i>	Untuk inisialisasi setiap nilai atribut yang akan di-assign datanya saat objek dibuat.
2	getKodeKereta()	<i>procedural</i>	Untuk pengembalian nilai atribut kode kereta yang akan memungkinkan kelas lain membaca nilai dari atribut private tanpa bisa mengubah nilainya secara langsung.
3	getNamaKereta()	<i>procedural</i>	Juga untuk pengembalian nilai atribut nama kereta (nilai huruf) yang berguna untuk baca saja tanpa bisa diubah nilainya.
4	getRutePerjalanan()	<i>procedural</i>	Untuk pengembalian nilai atribut rute perjalanan (nilai huruf) yang berguna untuk baca saja tanpa bisa diubah nilainya.
5	getSisaKursi()	<i>procedural</i>	Untuk pengembalian nilai atribut sisa kursi (nilai angka) yang berguna untuk baca saja tanpa bisa diubah nilainya.
6	setSisaKursi()	<i>procedural</i>	Untuk dapat mengubah nilai dari sisa kursi dan membuat atribut jadi dapat di-set nilainya secara publik. Juga diperuntukkan saat kursi dipesan, otomatis mengurangi kursi yang tersedia.
7	PusatKontrol()	<i>constructor</i>	Untuk inisialisasi atribut penyimpan objek kereta yang ditambahkan/disimpan ke

			dalam ArrayList sebagai kereta yang tersedia.
8	getDaftarK()	<i>procedural</i>	Untuk pengembalian nilai atribut daftarK (daftar kereta yang menyimpan setiap objek kereta setelah diinisialisasi) yang berguna untuk baca saja tanpa bisa diubah nilainya.
9	pemesanan()	<i>functional</i>	<i>Method</i> untuk melakukan pemesanan dengan setiap validasinya.
10	DataPenumpangTidakValidException()	<i>constructor</i>	<i>Unchecked Exception</i> yang mewarisi RuntimeException dan menginisialisasi pesan yang didapat sebagai cara menangani errornya.
11	RuteTidakDitemukanException()	<i>constructor</i>	<i>Checked Exception</i> yang mewarisi Exception dan menginisialisasi pesan yang didapat sebagai cara menangani errornya.
12	TiketHabisException()	<i>constructor</i>	<i>Checked Exception</i> yang mewarisi Exception dan menginisialisasi pesan secara langsung sebagai cara menangani errornya serta inisialisasi setiap atributnya.
13	getNamaKereta()	<i>procedural</i>	Untuk pengembalian nilai atribut nama kereta (nilai huruf) yang berguna untuk baca saja tanpa bisa diubah nilainya.
14	getSisaKursi()	<i>procedural</i>	Untuk pengembalian nilai atribut sisa kursi (nilai angka) yang berguna untuk baca saja tanpa bisa diubah nilainya.

IV. Dokumentasi dan Pembahasan Code

J KeretaApi.java > KeretaApi

```
public class KeretaApi {  
    private String kodeKereta;  
    private String namaKereta;  
    private String rutePerjalanan;  
    private int sisaKursi;  
  
    KeretaApi(String kodeKereta, String namaKereta, String rute, int sisaKursi) {  
        this.kodeKereta = kodeKereta;  
        this.namaKereta = namaKereta;  
        this.rutePerjalanan = rute;  
        this.sisaKursi = sisaKursi;  
    }  
  
    public String getKodeKereta() {  
        return kodeKereta;  
    }  
  
    public String getNamaKereta() {  
        return namaKereta;  
    }  
  
    public String getRutePerjalanan() {  
        return rutePerjalanan;  
    }  
  
    public int getSisaKursi() {  
        return sisaKursi;  
    }  
  
    // public void setKodeKereta(String kodeKereta) {  
    //     this.kodeKereta = kodeKereta;  
    // }  
  
    // public void setNamaKereta(String namaKereta) {  
    //     this.namaKereta = namaKereta;  
    // }  
  
    // public void setRutePerjalanan(String rutePerjalanan) {  
    //     this.rutePerjalanan = rutePerjalanan;  
    // }  
  
    public void setSisaKursi(int sisaKursi) {  
        this.sisaKursi -= sisaKursi;  
    }  
}
```

```

J PusatKontrol.java > PusatKontrol > pemesanan(String, String, String, int)
import java.util.ArrayList;

public class PusatKontrol {
    private ArrayList<KeretaApi> daftarK = new ArrayList<>();

    PusatKontrol() {
        daftarK.add(new KeretaApi(kodeKereta: "K01", namaKereta: "Argo Bromo", rute: "JKT - SBY", sisaKursi: 50));
        daftarK.add(new KeretaApi(kodeKereta: "K02", namaKereta: "Parahyangan", rute: "JKT - BOG", sisaKursi: 15));
    }

    public ArrayList<KeretaApi> getDaftarK() {
        return daftarK;
    }

    public void pemesanan(String kodeKereta, String nik, String namaP, int jumlahTiket) throws DataPenumpangTidakValidException, RuteTidakDitemukanException, TiketHabisException {
        if (nik.length() != 16) {
            throw new DataPenumpangTidakValidException("Data penumpang " + nik + " Tidak Valid!");
        }

        for (char c : nik.toCharArray()) {
            if (!Character.isDigit(c)) {
                throw new DataPenumpangTidakValidException("Data penumpang " + nik + " Tidak Valid!");
            }
        }

        KeretaApi valid = null;
        for (KeretaApi k : daftarK) {
            if (k.getKodeKereta().equals(kodeKereta)) {
                valid = k;
                break;
            }
        }

        if (valid == null) {
            throw new RuteTidakDitemukanException("Kode kereta '" + kodeKereta + "' tidak ditemukan dalam sistem.");
        }

        if (jumlahTiket > valid.getSisaKursi()) {
            throw new TiketHabisException(valid.getNamaKereta(), valid.getSisaKursi());
        }

        valid.setSisaKursi(jumlahTiket);
        System.out.println("\nRESERVASI BERHASIL DISIMPAN!");
        System.out.println("Penumpang : " + namaP);
        System.out.println("NIK : " + nik);
        System.out.println("Kereta : " + valid.getNamaKereta() + " (" + valid.getRutePerjalanan() + ")");
        System.out.println("Jumlah Tiket: " + jumlahTiket + " kursi");
    }
}

```

```

J DataPenumpangTidakValidException.java > DataPenumpangTidakValidException
public class DataPenumpangTidakValidException extends RuntimeException {
    public DataPenumpangTidakValidException(String pesan) {
        super(pesan);
    }
}

```

```

J RuteTidakDitemukanException.java > RuteTidakDitemukanException > RuteT
public class RuteTidakDitemukanException extends Exception {
    public RuteTidakDitemukanException(String pesan) {
        super(pesan);
    }
}

```

```
App.java > App > main(String[])  
import java.util.InputMismatchException;  
import java.util.Scanner;  
  
public class App {  
    Run | Debug  
    public static void main(String[] args) throws Exception {  
        Scanner in = new Scanner(System.in);  
        PusatKontrol kontrol = new PusatKontrol();  
        int pilih = 0;  
  
        System.out.println(x: "-----JAVA EXPRESS!-----");  
  
        do {  
            System.out.println(x: "\n--- MENU UTAMA JAVA EXPRESS ---");  
            System.out.println(x: "1. Lihat Jadwal & Sisa Kursi");  
            System.out.println(x: "2. Pesan Tiket");  
            System.out.println(x: "3. Keluar");  
            System.out.print(s: "Pilih: ");  
  
            try {  
                pilih = in.nextInt();  
                // in.nextInt();  
            }  
            catch (InputMismatchException e) {  
                System.out.println(x: "ERROR, Input harus berupa angka pilihan menu! Silakan coba lagi.");  
                in.nextLine();  
                continue;  
            }  
  
            switch (pilih) {  
                case 1:  
                    for (KeretaApi k : kontrol.getDaftarK()) {  
                        System.out.println("\nKODE\t| " + "NAMA KERETA\t|" + "RUTE \t|" + "SISA KURSI\t|");  
                        System.out.println(k.getKodeKereta() + "\t " + k.getNamaKereta() + "\t " + k.getRutePerjalanan() + "\t " + k.getSisaKursi());  
                    }  
                    break;  
  
                case 2:  
                    System.out.println(x: "\n--- FORM RESERVASI TIKET ---");  
                    System.out.print(s: "Masukkan Kode Kereta: ");  
                    in.nextLine();  
                    String kode = in.nextLine();  
  
                    System.out.print(s: "Masukkan NIK : ");  
                    String nik = in.nextLine();  
  
                    System.out.print(s: "Masukkan Nama : ");  
                    String nama = in.nextLine();
```

```

        int jumlahTiket = 0;
        System.out.print(s: "Jumlah Tiket      : ");

        try {
            jumlahTiket = in.nextInt();
            in.nextLine();
        }
        catch (InputMismatchException e) {
            System.out.println(x: "\nERROR, Jumlah tiket harus diisi dengan angka angka bulat!");
            in.nextLine();
            break;
        }

        try {
            kontrol.pemesanan(kode, nik, nama, jumlahTiket);
        }
        catch (DataPenumpangTidakValidException e) {
            System.out.println("\nVALIDASI GAGAL, " + e.getMessage());
        }
        catch (RuteTidakDitemukanException e) {
            System.out.println("\nROUTE ERROR, " + e.getMessage());
        }
        catch (TiketHabisException e) {
            System.out.println("\nKUOTA PENUH, " + e.getMessage());
        }
        break;

    case 3:
        System.out.println(x: "Bye");
        break;

    default:
        System.out.println(x: "Pilihan tidak valid! coba lagi");
        break;
    }

} while (pilih != 3);

try {
    System.out.println(x: "Menutup JAVA EXPRESS!");
}
finally {
    System.out.println(x: "Sayonara!");
    in.close();
}
}

```


-----JAVA EXPRESS!-----

--- MENU UTAMA JAVA EXPRESS ---

1. Lihat Jadwal & Sisa Kursi
2. Pesan Tiket
3. Keluar

Pilih: 1

KODE	NAMA KERETA	RUTE	SISA KURSI
K01	Argo Bromo	JKT - SBY	50

KODE	NAMA KERETA	RUTE	SISA KURSI
K02	Parahyangan	JKT - BDG	15

--- MENU UTAMA JAVA EXPRESS ---

1. Lihat Jadwal & Sisa Kursi
2. Pesan Tiket
3. Keluar

Pilih: 2

--- FORM RESERVASI TIKET ---

Masukkan Kode Kereta: K01
Masukkan NIK : 1234567890123456
Masukkan Nama : Dimas
Jumlah Tiket : 45

RESERVASI BERHASIL DISIMPAN!

Penumpang : Dimas
NIK : 1234567890123456
Kereta : Argo Bromo (JKT - SBY)
Jumlah Tiket: 45 kursi

--- MENU UTAMA JAVA EXPRESS ---

1. Lihat Jadwal & Sisa Kursi
2. Pesan Tiket
3. Keluar

Pilih: 2

--- FORM RESERVASI TIKET ---

Masukkan Kode Kereta: K01
Masukkan NIK : 123456789fufufafa
Masukkan Nama : fufu
Jumlah Tiket : 35

VALIDASI GAGAL, Data penumpang 123456789fufufafa Tidak Valid!

--- MENU UTAMA JAVA EXPRESS ---

1. Lihat Jadwal & Sisa Kursi
2. Pesan Tiket
3. Keluar

Pilih: 3

Bye

Menutup JAVA EXPRESS!

Sayonara!

A. Pembahasan Code

Pertama, dimulai dari *class* KeretaApi yang menjadi bagian utama atau blueprint dalam sistem pemesanan tiket kereta api. Deklarasi setiap atribut seperti kodeKereta sebagai kode yang bernilai huruf/karakter berupa String, namaKereta berupa String, rutePerjalanan yang juga berupa String karena menyimpan nilai rute dari perjalanan kereta, dan sisaKursi yang berupa integer. Penggunaan access modifier “private” diperlukan untuk menjaga atau mengenkapsulasi data atau nilainya dari *class-class* diluarnya. Melanjutkan konsep enkapsulasi, maka dibutuhkan *treatment* dengan getter sebagai pembaca nilainya dan setter yang dapat mengubah nilai dari atributnya secara public. Getter untuk setiap atributnya berfungsi untuk membaca dan mengambil nilainya, , setter pada sisaKursi (setSisaKursi()) diperuntukkan dalam mengurangi sisa kursi yang telah dipesan nantinya ketika dipanggil *method* diluar.

Selanjutnya dalam *class* PusatKontrol, deklarasikan collections berupa ArrayList untuk menyimpan kereta-kereta (objek kereta) yang tersedia, penggunaan ArrayList juga diperuntukkan agar objek yang disimpan dapat terurut. Kemudian dalam *constructor*-nya, melakukan penyimpanan objek kereta ke dalam ArrayList. Selanjutnya tidak lupa untuk memberikan getter pada collections yang dideklarasikan tersebut agar dapat membaca isi dari ArrayList tersebut. Lalu pembuatan *method* pemesanan() dengan penerapan throws DataPenumpangTidakValidException, RuteTidakDitemukanException, TiketHabisException. Validasi dalam *method*, dimulai dengan pengecekan NIK, jika nik tidak sama dengan 16 karakter/huruf akan dilemparkan error *Unchecked Exception* beserta pesannya untuk diinisialisasi pada *constructor* di *class* custom exception yaitu DataPenumpangTidakValidException yang mewarisi RuntimeException. Lalu validasi selanjutnya juga harus mengecek panjang nik-nya nanti jika mengandung huruf, penerapannya dengan perulangan foreach untuk menelusuri panjang huruf/karakternya, kemudian divalidasi jika karakter mengandung karakter selain angka (!Character.isDigit()) maka akan dilemparkan *Unchecked Exception* yang sama. Validasi selanjutnya untuk memeriksa, apakah objek keretanya ada? Dengan deklarasi objek KeretaApi dengan null dahulu, dan melalui perulangan foreach untuk menelusuri menggunakan tipe data KeretaApi (karena akan menelusuri objek) melalui penelusuran ArrayList daftarK. Validasi jika kode yang diinput sama dengan kodeKereta pada objek, maka akan disimpan ke dalam objek valid (tandanya kereta ketemu berdasarkan kodenya) dan perulangan berakhir. Selanjutnya validasi selanjutnya jika objek valid (objek kereta yang diperuntukkan untuk mencari) tetap null yang juga sudah mencakup kode dan rute pada objek, akan dilemparkan *Checked Exception* yang mewarisi Exception yaitu RuteTidakDitemukanException beserta pesan inisialisasinya. Lalu validasi terakhir, jika jumlahTiket yang dipesan lebih dari sisaKursi objek kereta (valid), maka melemparkan *Checked Exception* TiketHabisException yang mewarisi Exception beserta informasi nama kereta dan sisa kursi yang tersedia pada objek. Jika semua validasi dilewati dengan aman, maka perintahkan untuk mengurangi sisaKursi pada objek (valid) karena sudah dipesan.

Pada Custom Exception yang dibuat seperti *Unchecked Exception* DataPenumpangTidakValidException yang mewarisi RuntimeException hanya akan melemparkan pesan error begitu juga dengan *Checked Exception* yang mewarisi Exception yaitu RuteTidakDitemukanException. Dan untuk *Checked Exception* TiketHabisException yang mewarisi Exception perlu memberikan nilai saat inisialisasi

nama keretanya dan sisa kursinya. Terakhir dalam *class* Main, perlu deklarasi objek PusatKontrolnya terlebih dahulu sebagai alat kontrolnya sistem pemesanan tiket kereta api, melakukan do-while sebagai perulangan untuk selalu menjalankan terlebih dahulu setiap perintah di dalamnya selama bukan kondisi tertentu dan di dalamnya, memberikan Switch-case untuk menerapkan pemilihan menu yang pada case 2 menerapkan try-catch blok sebagai handling error dari ketika menginputkan jumlah tiket (jika berupa huruf, akan dilemparkan error default). Selanjutnya juga menerapkan multiple blok catch untuk menangani pemesanannya. Setelah keluar perulangan, juga menerapkan blok try yang tanpa catch, langsung dengan finally karena program telah selesai.

V. Kesimpulan

Program simulasi sistem pemesanan tiket kereta api ini, menghasilkan penerapan OOP (*Object-Oriented Programming*) yang juga menangani error saat program berjalan. Dalam penerapan *Unchecked Exception* *DataPenumpangTidakValidException* yang mewarisi *RuntimeException*, penerapan tersebut tidak wajib ditangani dan penggunaannya digunakan untuk saat program error. Sedangkan *Checked Exception* yang mewarisi *Exception* yaitu *RuteTidakDitemukanException* wajib ditangani oleh pemanggilnya yang berupa try-catch atau throws. Hasil dari penerapan keduanya, program dapat langsung melemparkan pesan error sesuai yang ingin dilemparkan ketika terjadi dengan throw pada PusatKontrol.

Pada program App (*class* App) penerapan dengan memanggil menggunakan try-catch yang dapat mencegah program crash terutama saat input, penerapan try-catch dalam case tersebut juga dapat memulihkan jalannya program ketika error dan melanjutkan eksekusinya. Terakhir penerapan blok finally diakhir baris program, dapat memisahkan logika penanganan error yang terjadi dalam switch-case dan perulangan, sehingga saat berakhir kode jadi lebih bersih.

VI. Daftar Pustaka

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). *Modul Praktikum Pemograman Berorientasi Objek; Error Handling*. Universitas Pertamina.

GeeksforGeeks. (2024, 21 April). *Error Handling in Programming*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/error-handling-in-programming/>.